Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 1

Группа: 241-327 Студент: Дубина Анастасия Александровна

Задание на лабораторную работу

Система Платежи. Клиент имеет Счет в банке и Кредитную Карту (КК). Клиент может оплатить Заказ, сделать платеж на другой Счет, заблокиро вать КК и аннулировать Счет. Администратор может заблокировать КК за превышение кредита.

Диаграмма классов

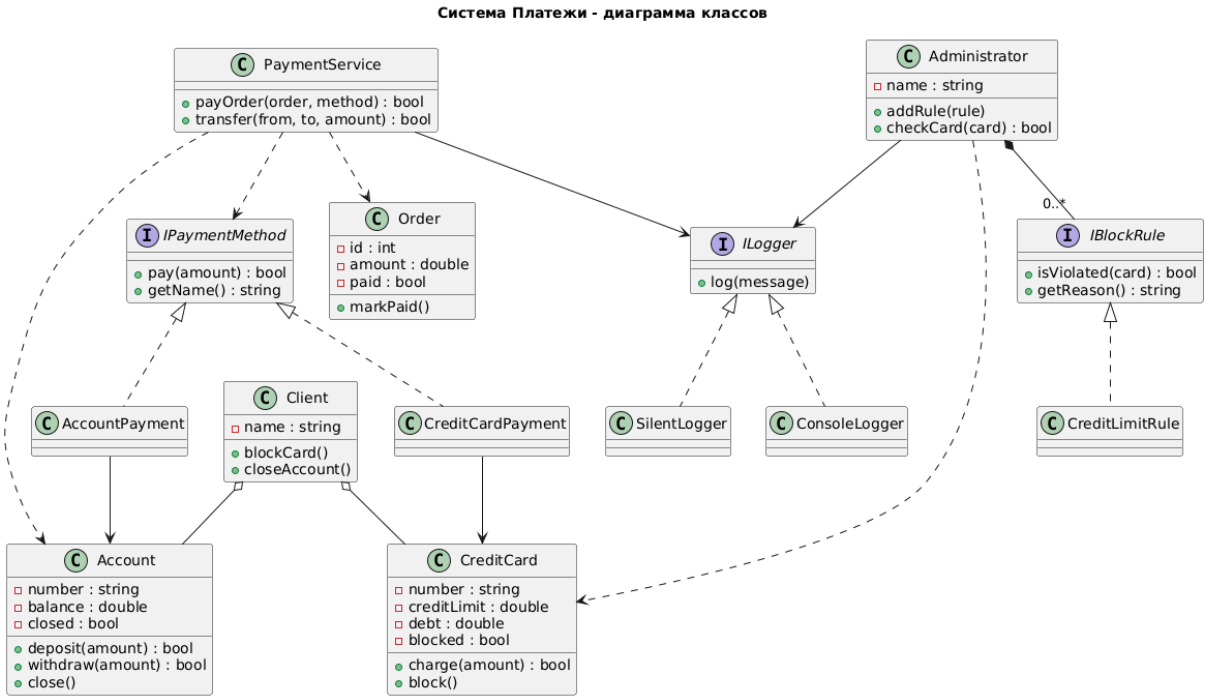
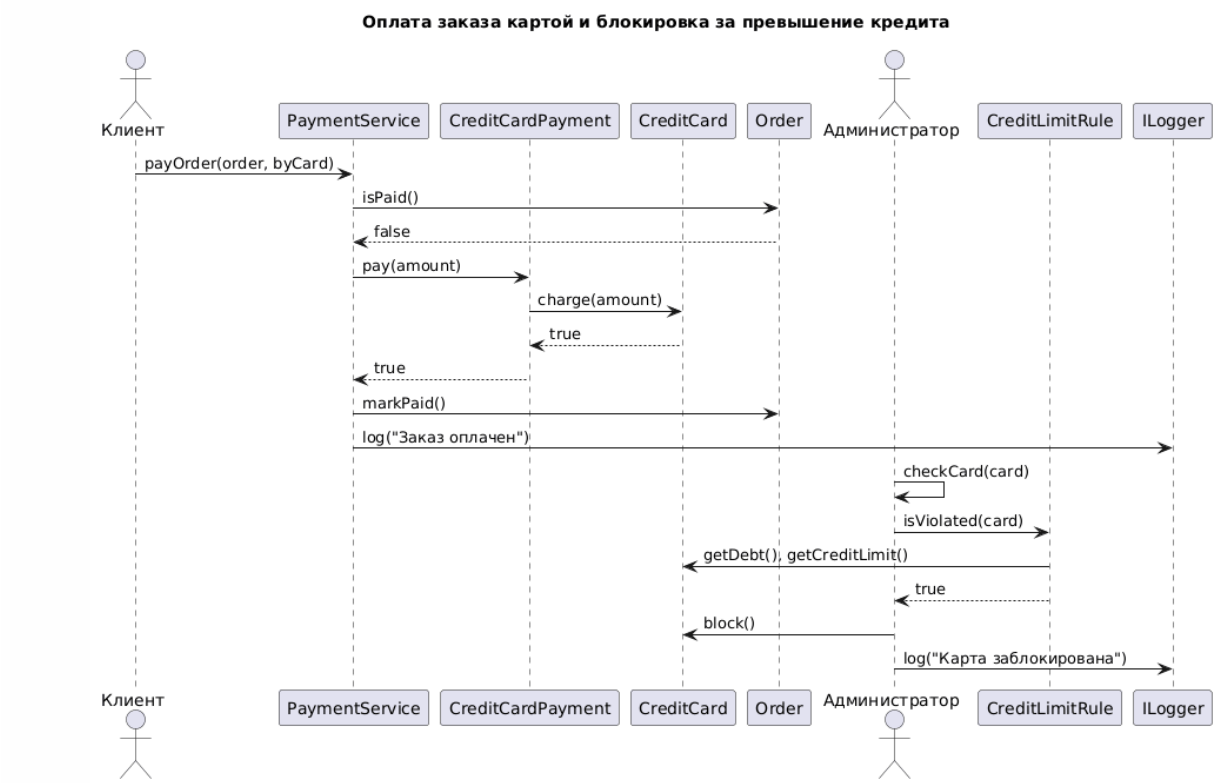


Диаграмма последовательности



Исходный код программы

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <clocale>
#include <sstream>
#include <iomanip>

using namespace std;

string money(double amount) {
   ostringstream out;
   out << fixed << setprecision(2) << amount;
   return out.str();
}
// ======================== Сущности предметной области ========================

// Счет в банке. Обязанность: хранить баланс и изменять его.
class Account {
public:
   Account(const string& number, double balance)
      : number(number), balance(balance), closed(false) {}
```

```
    string getNumber() const { return number; }
    double getBalance() const { return balance; }
    bool isClosed() const { return closed; }

    bool deposit(double amount) {
        if (closed || amount <= 0) return false;
        balance += amount;
        return true;
    }

    bool withdraw(double amount) {
        if (closed || amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    void close() { closed = true; }

private:
    string number;
    double balance;
    bool closed;
};

// Кредитная карта. Обязанность: хранить долг и состояние блокировки.
class CreditCard {
public:
    CreditCard(const string& number, double creditLimit)
        : number(number), creditLimit(creditLimit), debt(0), blocked(false) {}

    string getNumber() const { return number; }
    double getCreditLimit() const { return creditLimit; }
    double getDebt() const { return debt; }
    bool isBlocked() const { return blocked; }

    // Лимит здесь не проверяется: за превышение кредита карту блокирует Администратор
    bool charge(double amount) {
        if (blocked || amount <= 0) return false;
        debt += amount;
        return true;
    }

    void block() { blocked = true; }

private:
    string number;
```

```
    double creditLimit;
    double debt;
    bool blocked;
};

// Заказ. Обязанность: хранить сумму и признак оплаты.
class Order {
public:
    Order(int id, double amount) : id(id), amount(amount), paid(false) {}

    int getId() const { return id; }
    double getAmount() const { return amount; }
    bool isPaid() const { return paid; }
    void markPaid() { paid = true; }

private:
    int id;
    double amount;
    bool paid;
};

// Клиент. Обязанность: владеть своим счетом и картой.
class Client {
public:
    Client(const string& name, Account& account, CreditCard& card)
        : name(name), account(account), card(card) {}

    string getName() const { return name; }
    Account& getAccount() { return account; }
    CreditCard& getCard() { return card; }

    void blockCard() { card.block(); }
    void closeAccount() { account.close(); }

private:
    string name;
    Account& account;
    CreditCard& card;
};

// ======================== Вывод сообщений (SRP) ========================

// Вывод отделен от бизнес-логики: формат можно поменять, не трогая сервисы.
class ILogger {
public:
    virtual ~ILogger() {}
    virtual void log(const string& message) = 0;
```

```
};

class ConsoleLogger : public ILogger {
public:
   void log(const string& message) override {
      cout << "[LOG] " << message << endl;
   }
};

// ======================== Способы оплаты (OCP) ========================

// Новый способ оплаты добавляется новым классом-наследником,
// PaymentService при этом не изменяется.
class IPaymentMethod {
public:
   virtual ~IPaymentMethod() {}
   virtual bool pay(double amount) = 0;
   virtual string getName() const = 0;
};

class AccountPayment : public IPaymentMethod {
public:
   AccountPayment(Account& account) : account(account) {}
   bool pay(double amount) override { return account.withdraw(amount); }
   string getName() const override { return "счет " + account.getNumber(); }

private:
   Account& account;
};

class CreditCardPayment : public IPaymentMethod {
public:
   CreditCardPayment(CreditCard& card) : card(card) {}
   bool pay(double amount) override { return card.charge(amount); }
   string getName() const override { return "карта " + card.getNumber(); }

private:
   CreditCard& card;
};

// ======================== Сервис платежей ========================

// Обязанность: проводить операции оплаты и переводов.
class PaymentService {
public:
   PaymentService(ILogger& logger) : logger(logger) {}
```

```
    bool payOrder(Order& order, IPaymentMethod& method) {
        if (order.isPaid()) {
            logger.log("Заказ " + to_string(order.getId()) + " уже оплачен");
            return false;
        }
        if (!method.pay(order.getAmount())) {
            logger.log("Не удалось оплатить заказ " + to_string(order.getId()) +
                       " (" + method.getName() + ")");
            return false;
        }
        order.markPaid();
        logger.log("Заказ " + to_string(order.getId()) + " оплачен (" + method.getName() + ")");
        return true;
    }

    bool transfer(Account& from, Account& to, double amount) {
        if (to.isClosed()) {
            logger.log("Перевод невозможен: счет " + to.getNumber() + " закрыт");
            return false;
        }
        if (!from.withdraw(amount)) {
            logger.log("Перевод невозможен: недостаточно средств или счет " +
                       from.getNumber() + " закрыт");
            return false;
        }
        to.deposit(amount);
        logger.log("Перевод " + money(amount) + " со счета " + from.getNumber() +
                   " на счет " + to.getNumber());
        return true;
    }

private:
    ILogger& logger;
};

// ======================== Правила блокировки карт (OCP) ========================

// Новое правило блокировки = новый класс, Administrator не изменяется.
class IBlockRule {
public:
    virtual ~IBlockRule() {}
    virtual bool isViolated(const CreditCard& card) const = 0;
    virtual string getReason() const = 0;
};

class CreditLimitRule : public IBlockRule {
public:
```

```
    bool isViolated(const CreditCard& card) const override {
        return card.getDebt() > card.getCreditLimit();
    }
    string getReason() const override { return "превышение кредита"; }
};

// Администратор. Обязанность: проверять карты по правилам и блокировать нарушителей.
class Administrator {
public:
    Administrator(const string& name, ILogger& logger) : name(name), logger(logger) {}

    void addRule(unique_ptr<IBlockRule> rule) { rules.push_back(move(rule)); }

    bool checkCard(CreditCard& card) {
        for (const auto& rule : rules) {
            if (rule->isViolated(card)) {
                card.block();
                logger.log("Администратор " + name + " заблокировал карту " +
                           card.getNumber() + ": " + rule->getReason());
                return true;
            }
        }
        return false;
    }

private:
    string name;
    ILogger& logger;
    vector<unique_ptr<IBlockRule>> rules;
};

// ======================== Тестирование ========================

// Пустой логгер, чтобы тесты не засоряли вывод.
class SilentLogger : public ILogger {
public:
    void log(const string&) override {}
};

class Tester {
public:
    Tester() : passed(0), failed(0) {}

    void check(bool condition, const string& name) {
        if (condition) {
            passed++;
            cout << "  [OK]   " << name << endl;
```

```
        } else {
            failed++;
            cout << "  [FAIL] " << name << endl;
        }
    }

    void printResult() const {
        cout << "Пройдено: " << passed << ", провалено: " << failed << endl;
    }

private:
    int passed;
    int failed;
};

void testAccount(Tester& t) {
    Account acc("A1", 100);
    t.check(acc.deposit(50) && acc.getBalance() == 150, "Пополнение счета");
    t.check(acc.withdraw(30) && acc.getBalance() == 120, "Списание со счета");
    t.check(!acc.withdraw(500) && acc.getBalance() == 120, "Нельзя списать больше баланса");
    acc.close();
    t.check(acc.isClosed() && !acc.deposit(10), "Закрытый счет не пополняется");
}

void testCreditCard(Tester& t) {
    CreditCard card("C1", 1000);
    t.check(card.charge(200) && card.getDebt() == 200, "Оплата картой увеличивает долг");
    card.block();
    t.check(card.isBlocked() && !card.charge(10), "Заблокированной картой нельзя платить");
}

void testPayOrder(Tester& t) {
    SilentLogger logger;
    PaymentService service(logger);
    Account acc("A1", 500);
    CreditCard card("C1", 1000);
    AccountPayment byAccount(acc);
    CreditCardPayment byCard(card);

    Order o1(1, 300);
    t.check(service.payOrder(o1, byAccount) && o1.isPaid() && acc.getBalance() == 200,
        "Оплата заказа со счета");
    t.check(!service.payOrder(o1, byAccount), "Повторная оплата заказа запрещена");

    Order o2(2, 400);
    t.check(!service.payOrder(o2, byAccount) && !o2.isPaid(),
        "Оплата со счета при нехватке средств");
```

```
    t.check(service.payOrder(o2, byCard) && card.getDebt() == 400, "Оплата заказа картой");
}

void testTransfer(Tester& t) {
    SilentLogger logger;
    PaymentService service(logger);
    Account from("A1", 500);
    Account to("A2", 0);

    t.check(service.transfer(from, to, 200) && from.getBalance() == 300 && to.getBalance() == 200,
        "Перевод на другой счет");
    to.close();
    t.check(!service.transfer(from, to, 100) && from.getBalance() == 300,
        "Перевод на закрытый счет запрещен");
}

void testClient(Tester& t) {
    Account acc("A1", 100);
    CreditCard card("C1", 500);
    Client client("Иванов", acc, card);
    client.blockCard();
    t.check(card.isBlocked(), "Клиент блокирует свою карту");
    client.closeAccount();
    t.check(acc.isClosed(), "Клиент аннулирует счет");
}

void testAdministrator(Tester& t) {
    SilentLogger logger;
    Administrator admin("Петров", logger);
    admin.addRule(unique_ptr<IBlockRule>(new CreditLimitRule()));

    CreditCard card("C1", 1000);
    card.charge(800);
    t.check(!admin.checkCard(card) && !card.isBlocked(), "Карта в пределах лимита не блокируется");
    card.charge(300);
    t.check(admin.checkCard(card) && card.isBlocked(), "Администратор блокирует карту за превышение кредита");
}

void runTests() {
    Tester t;
    cout << "\n===== Тестирование =====" << endl;
    testAccount(t);
    testCreditCard(t);
    testPayOrder(t);
    testTransfer(t);
    testClient(t);
    testAdministrator(t);
```

```
    t.printResult();
}

// ======================== Демонстрация ========================

int main() {
    setlocale(LC_ALL, "Russian");

    ConsoleLogger logger;
    PaymentService service(logger);
    Administrator admin("Петров", logger);
    admin.addRule(unique_ptr<IBlockRule>(new CreditLimitRule()));

    Account ivanovAcc("40817-001", 10000);
    CreditCard ivanovCard("4276-1111", 5000);
    Client ivanov("Иванов", ivanovAcc, ivanovCard);

    Account sidorovAcc("40817-002", 2000);
    CreditCard sidorovCard("4276-2222", 3000);
    Client sidorov("Сидоров", sidorovAcc, sidorovCard);

    cout << "===== Демонстрация =====" << endl;

    Order order1(1, 3000);
    AccountPayment byAccount(ivanov.getAccount());
    service.payOrder(order1, byAccount);

    Order order2(2, 6000);
    CreditCardPayment byCard(ivanov.getCard());
    service.payOrder(order2, byCard);
    admin.checkCard(ivanov.getCard());

    Order order3(3, 100);
    service.payOrder(order3, byCard);

    service.transfer(ivanov.getAccount(), sidorov.getAccount(), 1500);

    sidorov.blockCard();
    logger.log("Клиент " + sidorov.getName() + " заблокировал свою карту");
    sidorov.closeAccount();
    logger.log("Клиент " + sidorov.getName() + " аннулировал счет");
    service.transfer(ivanov.getAccount(), sidorov.getAccount(), 500);

    runTests();
    return 0;
}
```